TI102 - Algorithmique

Barème et corrigé du DE du 23/11/2024

Compréhension (8 points)

Recherche dichotomique (4 points)

Exercice 1. La recherche dichotomique est un algorithme qui trouve efficacement une valeur dans un tableau trié.

```
Algorithme : recherche_dichotomique
Variables :
  l, r, m : entier
  iter, pos, val : entier
  T[10] : tableau d'entier
Début
 1:   T ← {1, 1, 2, 3, 5, 8, 13, 21, 34, 55}
 2:   Lire(val)
 3:   pos ← −1
 4:   iter ← 0
 5:   l ← 1
 6:   r ← 10
 7:   Tant que l ≤ r  ET pos = −1  Faire
 8:       iter ← iter + 1
 9:       m ← l + (r − l)/2
10:       Si T[m] = val  Alors
11:           pos ← m
12:       Sinon
13:           Si T[m] < val  Alors
14:               l ← m + 1
15:           Sinon
16:               r ← m − 1
17:           Fin Si
18:       Fin Si
19:   Fin Tant que
20:   Si pos ≠ −1  Alors
21:       Ecrire("Valeur trouvée en ", iter, "itérations\NL")
22:   Fin Si
Fin
```

En supposant qu'à la ligne 2, la valeur lue est 13, représenter la trace d'exécution de l'algorithme ci-dessus en remplissant le tableau suivant :

ligne	val	pos	iter	l	r	m
init	?	?	?	?	?	?
2	13					
3		-1				
4			0			
5				1		
6					10	
⋮	⋮	⋮	⋮	⋮	⋮	⋮

Solution: **[Barème]** *4 points au total: 1 point pour chaque itération correctement remplie et -0.25 par erreur.*

ligne	val	pos	iter	l	r	m	Sortie
init	?	?	?	?	?	?	
2	13						
3		-1					
4			0				
5				1			
6					10		
8			1				
9						5	
14				6			
8			2				
9						8	
16					7		
8			3				
9						6	
14				7			
8			4				
9						7	
11		7					
21							Valeur trouvée en 4 itérations

■

La bonne sortie (2 points)

Exercice 2. On considère les trois fonctions `f`, `g` et `h` dont les définitions sont les suivantes. Quelle est la sortie de l'algorithme `main` ?

```
Fonction : f(x: entier) → entier
Variables locales :
Début
  Retourner x * x
Fin
```

```
Fonction : g(x: référence entier)
Variables locales :
Début
  x ← x − 7
Fin
```

```
Fonction : h(x: référence entier)
Variables locales :
  y : entier
Début
  y ← f(x)
  g(&y)
  x ← y
Fin
```

```
Algorithme : main
Variables locales :
  a, b : entier
Début
  a ← 4
  Ecrire(f(a), "\NL")
  g(&a)
  Ecrire(a, "\NL")
  b ← 7
  h(&b)
  Ecrire(b, "\NL")
Fin
```

Solution: **[Barème]** *2 points au total: 0.5 point les deux premiers affichages, 1 point pour le troisième.* Voici la sortie de l'algorithme `main` :

```
16
-3
42
```

■

Matrice mystère (2 points)

Exercice 3. Parmi les propositions suivantes, quelle est celle qui correspond à la représentation matricielle du tableau 2D M à la fin de l'exécution l'algorithme `matrice_mystere` ci-dessous ?

```
Algorithme : matrice_mystere
Variables :
  M[5][5] : tableau d'entier
  i, j, n : entier
Début
  n ← 5
  Pour i ← 1 à n Faire
      Pour j ← 1 à n Faire
          Si j ≥ n − i + 1 Alors
              M[i][j] ← 1
          Sinon
              M[i][j] ← 0
          Fin Si
      Fin Pour
  Fin Pour
Fin
```

(a)

1	1	1	1	0
1	1	1	0	0
1	1	0	0	0
1	0	0	0	0
0	0	0	0	0

(b)

1	1	1	1	1
0	1	1	1	1
0	0	1	1	1
0	0	0	1	1
0	0	0	0	1

(c)

0	0	0	0	0
1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0

(d)

0	0	0	0	1
0	0	0	1	1
0	0	1	1	1
0	1	1	1	1
1	1	1	1	1

Solution: **[Barème]** *2 points.* La bonne réponse est la proposition (d). ■

Codage (12 points)

Le gène Z (3 points)

Exercice 4. Une séquence d'ADN est une suite finie de nucléotides que l'on représente par les lettres `A`, `C`, `G` et `T`. Un gène est une séquence d'ADN spécifique qui encode une information génétique. Un gène est défini par un motif de nucléotides qui se répète (ou pas) dans une séquence d'ADN donnée.

Par exemple, dans la séquence d'ADN `"ACGTACGTACGTACGT"`, le gène dont le motif est `"ACGT"` est répété 4 fois. Tandis que le gène dont le motif est `"AAGT"` n'est pas présent dans la séquence.

Écrire une fonction `genz(adn: chaîne de caractères, gene: chaîne de caractères) → entier` qui prend en paramètres une séquence d'ADN `adn` et un motif de gène `gene` et qui renvoie le nombre de fois que le motif `gene` est répété dans la séquence `adn`.

Exemple : `genz("ACGTACGTACGTACGT", "ACGT")` *doit renvoyer 4 tandis que* `genz("ACGTACGTACGTACGT", "AAGT")` *doit renvoyer 0.*

Solution: **[Barème]** *3 points au total: si trop d'erreur de syntaxe retirez jusqu'à 1 point sur le total. Si la logique est correcte mais erreurs d'implémentation mineurs (mauvais indices, incrémentation incorrecte, variables non déclarées, etc.) jusqu'à 1.5 point sur le total. Pas plus de 1.5 point pour une logique incorrecte.*

Voici une solution possible :

```
Fonction : genz(adn: chaîne de caractères, gene: chaîne de caractères) → entier
Variables locales :
  i, j, n, m, compteur : entier
Début
  n ← longueur(adn)
  m ← longueur(gene)
  compteur ← 0
  i ← 1
  Tant que i ≤ n − m + 1 Faire
      j ← 1
      Si adn[i] = gene[j] Alors
          Tant que j ≤ m ET adn[i + j − 1] = gene[j] Faire
              j ← j + 1
          Fin Tant que
          Si j > m Alors
              compteur ← compteur + 1
          Fin Si
      Fin Si
      i ← i + 1
  Fin Tant que
  Retourner compteur
Fin
```

■

Scrutin de Condorcet (9 points)

Exercice 5. Le scrutin de Condorcet est un système de vote où chaque électeur classe les candidats dans l'ordre de leurs préférence. Le vainqueur, s'il existe, est le candidat qui, comparé tour à tour à chacun des autres candidats, s'avère à chaque fois être le candidat préféré. Dans un scrutin à n candidats, chaque candidat est représenté par un entier de 1 à n et un bulletin de vote est alors une liste d'entiers ordonnée par ordre de préférence.

Par exemple, pour un scrutin à 3 candidats et 2 votants, le bulletin $B_i = \{3,1,2\}$ signifie que le candidat 3 est préféré au candidat 1, puis à son tour, le candidat 1 est préféré au candidat 2. Et le bulletin $B_j = \{3,2,1\}$ signifie que le candidat 3 est préféré au candidat 2 qui est préféré au candidat 1. Dans ce cas, le candidat 3 est le gagnant du scrutin de Condorcet car dans les deux bulletins, il est préféré à chacun des autres candidats.

Dans cet exercice un bulletin est représenté par un tableau d'entiers de taille n et un scrutin est représenté par un tableau de bulletins.

Voici les consignes générales pour la correction de ce problème :
- Jusqu'à −25% de la note si trop d'erreurs de syntaxe.
- Jusqu'à −25% de la note si pas d'appel des fonctions précédentes.
- Immédiatement −25% de la note si l'en-tête de la fonction n'est pas respectée (elle est donnée dans l'énoncé).
- Pas plus de 50% de la note si la logique est incorrecte.

1. Un bulletin est dit *valide* pour un nombre n de candidats si tous les candidats numérotés de 1 à n sont représentés exactement une fois dans le bulletin.

   (a) Écrire une fonction `differents(b: tableau d'entier, n: entier) → booléen` qui prend en paramètres un tableau d'entiers `b` de taille `n` et renvoie `Vrai` si tous les éléments de `b` sont différents, et renvoie `Faux` sinon.

   Exemple : `differents({3,1,2}, 3)` *renvoie* `Vrai` *et* `differents({3,1,3}, 3)` *renvoie* `Faux`.

   Solution: [Barème] *1 point*

```
Fonction : differents(b: tableau d'entier, n: entier) → booléen
Variables locales :
  i, j : entier
Début
  Pour i ← 1 à n Faire
      Pour j ← i+1 à n Faire
          Si b[i] = b[j] Alors
              Retourner Faux
          Fin Si
      Fin Pour
  Fin Pour
  Retourner Vrai
Fin
```

■

   (b) Écrire une fonction `domaine(b: tableau d'entier, n: entier) → booléen`, qui prend en paramètres un tableau d'entiers `b` de taille `n` et renvoie `Vrai` si tous les éléments de `b` sont compris

entre 1 et `n`, et renvoie `Faux` sinon.

Exemple : `domaine({3,1,2}, 3)` *renvoie* `Vrai` *et* `domaine({3,1,4}, 3)` *renvoie* `Faux`.

Solution: [Barème] *1 point*

```
Fonction : domaine(b: tableau d'entier, n: entier) → booléen
Variables locales :
  i : entier
Début
  Pour i ← 1 à n Faire
      Si b[i] < 1 OU b[i] > n Alors
          Retourner Faux
      Fin Si
  Fin Pour
  Retourner Vrai
Fin
```

■

(c) En vous aidant des deux fonctions précédentes, écrire une fonction `est_valide(b: tableau d'entier, n: entier) → booléen` qui prend en paramètres un bulletin `b` et le nombre de candidats `n`, et qui renvoie `Vrai` si le bulletin est valide, et renvoie `Faux` sinon.

Exemple : `est_valide({3,1,2}, 3)` *renvoie* `Vrai` *et* `est_valide({3,1,3}, 3)` *renvoie* `Faux`.

Solution: [Barème] *0.5 point*

```
Fonction : est_valide(b: tableau d'entier, n: entier) → booléen
Variables locales :
  i : entier
Début
  Retourner differents(b,n) ET domaine(b,n)
Fin
```

■

2. On rappelle qu'un bulletin représente les candidats dans un ordre de *préférence*, c'est-à-dire que dans un bulletin B, le candidat i est préféré au candidat j si i paraît avant j dans B. Ensuite le décompte de voix se fait en comptant le nombre de bulletins qui préfèrent un candidat à un autre.

   Par exemple, pour un scrutin à 3 candidats et 3 bulletins : $B_1 = \{3,1,2\}$, $B_2 = \{3,2,1\}$ et $B_3 = \{3,1,2\}$, le décompte des voix est fait dans le tableau `resultats` suivant.

	1	2	3
1	0	2	0
2	1	0	0
3	3	3	0

Dans ce tableau, `resultats[i][j]` contient le nombre de bulletins qui préfèrent le candidat i au candidat j. En effet, tous les bulletins préfèrent le candidat 3 à tous les autres candidats, donc `resultats[3][1] = 3`, `resultats[3][2] = 3`. Les bulletins B_1 et B_3 préfèrent le candidat 1 au candidat 2, donc `resultats[1][2] = 2`. Et le bulletin B_2 préfère le candidat 2 au candidat 1, donc `resultats[2][1] = 1`.

Dans toute la suite du problème on supposera qu'il y a au plus 20 candidats lors du scrutin.

(a) Écrire une fonction `prefere(b: tableau d'entier, n: entier, i: entier, j: entier) → booléen` qui prend en paramètres un bulletin `b`, le nombre de candidats `n`, et deux candidats `i` et `j`, et qui renvoie `Vrai` si le bulletin est **valide** et que le candidat `i` est préféré au candidat `j` dans ce bulletin, et renvoie `Faux` sinon.

Exemple : `prefere({3,1,2}, 3, 3, 1)` *renvoie* `Vrai` *et* `prefere({3,1,2}, 3, 2, 1)` *renvoie* `Faux`. *Puis* `prefere({3,1,1}, 3, 3, 1)` *renvoie* `Faux` *car le bulletin n'est pas valide.*

Solution: [Barème] *1 point*

```
Fonction : prefere(b: tableau d'entier, n: entier, i: entier, j: entier) → booléen
Variables locales :
  k, pos_i, pos_j : entier
Début
  Pour k ← 1 à n Faire
      Si b[k] = i Alors
          pos_i ← k
      Fin Si
      Si b[k] = j Alors
          pos_j ← k
      Fin Si
  Fin Pour
  Retourner est_valide(b,n) ET pos_i < pos_j
Fin
```

■

(b) Écrire une fonction `decompte(scrutin: tableau d'entier, nb: entier, n: entier) → tableau d'entier` qui prend en paramètres un tableau de bulletins `scrutin`, le nombre de bulletins `nb`, et le nombre de candidats `n`, et qui renvoie un tableau d'entier à deux dimensions `resultats` tel que `resultats[i][j]` contient le nombre de bulletins qui préfèrent le candidat `i` au candidat `j`.

Exemple : `decompte({{3,1,2}, {3,2,1}, {3,1,2}}, 3, 3)` *renvoie le tableau* `{{0,2,0}, {1,0,0}, {3,3,0}}`.

Solution: [Barème] *2.5 points*

```
Fonction : decompte(scrutin: tableau d'entier, nb: entier, n: entier)
→ tableau d'entier
Variables locales :
  resultats[20][20] : tableau d'entier (* On suppose que n ≤ 20 *)
  i, j, k : entier
Début
  Pour i ← 1 à n Faire
      Pour j ← 1 à n Faire
          resultats[i][j] ← 0
      Fin Pour
  Fin Pour
  Pour k ← 1 à nb Faire
      Pour i ← 1 à n Faire
          Pour j ← 1 à n Faire
              Si prefere(scrutin[k], n, i, j) Alors
                  resultats[i][j] ← resultats[i][j] + 1
              Fin Si
          Fin Pour
      Fin Pour
  Fin Pour
  Retourner resultats
Fin
```

■

3. Un candidat i est le *gagnant* d'un scrutin de Condorcet si pour chaque autre candidat j, le nombre de bulletins qui préfèrent le candidat i au candidat j est strictement supérieur au nombre de bulletins qui préfèrent le candidat j au candidat i.

   Il est possible qu'un scrutin de Condorcet n'a pas de gagnant, mais si il existe un gagnant alors il est unique.

   (a) Écrire une fonction `est_gagnant(scrutin: tableau d'entier, nb: entier, n: entier, i: entier) → booléen` qui prend en paramètres un tableau de bulletins `scrutin`, le nombre de bulletins `nb`, le nombre de candidats `n`, ainsi que le numéro `i` d'un candidat et vérifie si le candidat `i` est vainqueur du scrutin. **Cette fonction effectue d'abord le décompte des voix**, puis renvoie `Vrai` si le candidat `i` est le gagnant du scrutin, et renvoie `Faux` sinon.

   Exemple : *Pour le scrutin de l'exemple précédent,* `est_gagnant({{3,1,2}, {3,2,1}, {3,1,2}}, 3, 3, 3)` *renvoie* `Vrai` *car le candidat* 3 *est préféré à chacun des autres candidats. Et* `est_gagnant({{3,1,2}, {3,2,1}, {3,1,2}}, 3, 3, 1)` *renvoie* `Faux` *car même si le candidat* 1 *est préféré au candidat* 2*, il n'est pas préféré au candidat* 3.

   Solution: **[Barème]** *2 points*

```
Fonction : est_gagnant(scrutin: tableau d'entier, nb: entier, n: entier, i: entier) → b
Variables locales :
  resultats[20][20] : tableau d'entier (* On suppose que n ≤ 20 *)
  j : entier
Début
  resultats ← decompte(scrutin, nb, n)
  Pour j ← 1 à n Faire
      Si i ≠ j ET resultats[i][j] ≤ resultats[j][i] Alors
          Retourner Faux
      Fin Si
  Fin Pour
  Retourner Vrai
Fin
```

■

(b) En vous aidant de toutes les fonctions précédentes, écrire une fonction `gagnant(scrutin: tableau d'entier, nb: entier, n: entier) → entier` qui renvoie le numéro du candidat gagnant du scrutin de Condorcet à `nb` votants et `n` candidats s'il existe. La fonction renvoie -1 sinon.

Exemple : `gagnant({{3,1,2}, {3,2,1}, {3,1,2}}, 3, 3)` *renvoie* 3 *car le candidat* 3 *est préféré à chacun des autres candidats, de plus dans un scrutin de Condorcet, s'il y a un gagnant, il est nécessairement unique.*

Solution: [Barème] *1 point*

```
Fonction : gagnant(scrutin: tableau d'entier, nb: entier, n: entier) → entier
Variables locales :
  i : entier
Début
  Pour i ← 1 à n Faire
      Si est_gagnant(scrutin, nb, n, i) Alors
          Retourner i
      Fin Si
  Fin Pour
  Retourner -1
Fin
```

■